

Table des matières

Cours 04 — Lire une base et une API.....	1
Objectifs.....	1
1. Tirer ou recevoir : MQTT, REST et ODBC.....	2
2. Lire une base par ODBC	2
3. La capture de changements.....	4
4. Tirer une API REST.....	5
Points clés à retenir.....	7
Exercices.....	7

Cours 04 — Lire une base et une API

Théorie · Module 4/12 — Formation Data Engineering · ~20 min · intermédiaire

Où vous êtes : source → **ingestion** → lac → entrepôt → restitution. Le Cours 04 va chercher la donnée là où elle ne se livre pas en fichiers : dans la base de l'ERP, et dans l'API météo d'ECCC.

En 30 secondes :

- Une source qui pousse livre ce qu'elle a, quand elle l'a ; une source qu'on tire attend la demande. La base et l'API se tirent : c'est vous qui savez quoi demander, et quand.
- Un pilote comme ODBC fait de l'accès à une base un contrat, et une base de production se lit avec des égards : par paquets, hors des pointes, avec un compte en lecture.
- On ne relit pas une base entière chaque nuit : on capte ce qui a changé, par un filigrane sur une date de modification, par des documents de modification ou par le journal de la base.
- Une API se lit page par page, jusqu'à la dernière, avec des reprises bornées, et l'on compte ce qu'on a reçu.

L'idée du cours. Au Cours 03, les fichiers livrés sont entrés dans la RAW. Ici, on lit la base de l'ERP par ODBC, pour prouver que les extraits racontent la même histoire qu'elle, et la vraie API météo d'ECCC, page par page. Le poste s'agrandit : Ubuntu, Docker et le pilote ODBC entrent en scène.

Objectifs

- Situer une source qui pousse et une source qu'on tire, et ce que chacune laisse à votre charge.
- Lire une base relationnelle par ODBC sans la mettre en danger.
- Capturer les changements d'une base par requête, et connaître les limites de la méthode.
- Tirer une API REST paginée avec des reprises, et prouver qu'on a tout reçu.

1. Tirer ou recevoir : MQTT, REST et ODBC

Labo : Labo_04.md, étapes 2 et 4.

Le Cours 03 a chargé la voie la plus répandue, le dépôt de fichiers, que la source écrit et que vous ramassez. Les deux autres voies se tirent : la base, par un pilote, et l'API, page par page.

MQTT, la voie de la télémétrie en direct, suit une autre logique, et le Cours 09 la traite. À côté des deux voies tirées de ce cours, elle se situe ainsi :

	MQTT	REST	ODBC
Qui prend l'initiative	la source publie, l'abonné reçoit	le client demande	le client interroge
Ce qui circule	des messages courts, rangés par sujet	des documents, souvent en JSON, sur HTTP	des lignes de tables, en réponse à du SQL
Garantie de livraison	trois niveaux de qualité de service, dont « au moins une fois », qui produit des doublons	aucune au-delà de la réponse : le client réessaie	celle de la base : une requête voit un état cohérent
Usage type	des capteurs et des véhicules qui publient en continu	des services ouverts, des applications infonuagiques	les bases d'entreprise, l'ERP
Chez Boréal	la télémétrie et l'usine en direct (Cours 09)	l'API d'ECCE	la base de l'ERP

À retenir — tirer ou recevoir. Un système qui pousse (MQTT) livre ce qu'il a, quand il l'a : il faut être là pour le recevoir. Un système qu'on tire (REST, ODBC) attend la demande : c'est vous qui savez quoi demander, et quand. Presque tous les pièges d'une ingestion viennent de ce partage des responsabilités.

2. Lire une base par ODBC

Labo : Labo_04.md, étapes 1 et 2.

ODBC (*Open Database Connectivity*) est une interface standard, née au début des années 1990, qui permet à un programme de parler à une base par le même jeu d'appels, quel que soit son moteur. Quatre pièces se passent la requête :

Pièce	Ce qu'elle fait	Dans le labo
l'application	ouvre une connexion, envoie du SQL, lit les lignes	<code>extraire_erp_odbc.py</code> , avec <code>pyodbc</code>
le gestionnaire de pilotes	trouve le pilote demandé et lui passe les appels	celui de Windows ; unixODBC sous Linux
le pilote	traduit les appels ODBC dans le protocole de la base	psqlODBC, le pilote de PostgreSQL
la base	exécute la requête	PostgreSQL 18, dans son conteneur, sur le port 5433

La connexion se décrit par une chaîne : le pilote, le serveur, le port, la base, l'utilisateur et le mot de passe. Un nom de source de données (*DSN*), enregistré dans Windows, peut en tenir lieu ; une chaîne complète a l'avantage de voyager avec le code. En Python, `pyodbc` suit l'interface standard des bases de données du langage (DB-API 2.0) : `connect`, `cursor`, `execute`, `fetchmany`. C'est la même que celle du paquet `duckdb` ou de `psycopg`. En Java, JDBC (Java Database Connectivity) joue le rôle d'ODBC.

Le pilote est un contrat. Le code ne sait rien du protocole de PostgreSQL : changer de base, c'est changer de pilote et de chaîne de connexion, et parfois de dialecte SQL. C'est ce qui permet à un même outil de lire SAP HANA, SQL Server ou Oracle en entreprise.

Lire une base de production demande des égards. Une extraction qui balaie VBAP consomme les disques et la mémoire dont les ventes ont besoin, et, dans PostgreSQL, une lecture longue retient l'ancienne version des lignes modifiées pendant qu'elle dure. D'où les usages du métier :

Usage	Ce qu'il évite
extraire hors des heures de pointe, ou sur une réplique en lecture	de ralentir ceux qui saisissent les commandes
un compte qui n'a que le droit de lire	qu'une erreur de script modifie la base
lire par paquets (<code>fetchmany</code>)	de tenir une table entière en mémoire
ne lire que les lignes changées (section 3) et les colonnes utiles	de relire chaque nuit ce qui n'a pas bougé

Dans le labo, la base tourne dans un conteneur de l'Ubuntu de WSL 2 et publie le port 5433 ; Windows la joint par `localhost:5433`, parce que WSL relaie les ports ouverts dans Ubuntu. Le 5432 reste libre pour un PostgreSQL déjà installé sur le poste.

Piège — Ubuntu qui s'arrête dans votre dos. WSL arrête Ubuntu peu après la fermeture du dernier terminal Ubuntu, et Docker s'arrête avec elle. Le conteneur passe à l'état `Exited`, et le script Windows qui voulait lire la base reçoit une connexion refusée. Le délai se règle par `instanceIdleTimeout`, dans

%UserProfile%\wslconfig : c'est l'étape 6 du guide d'installation du parcours. À défaut du réglage, gardez un terminal Ubuntu ouvert pendant que vous travaillez avec la base.

3. La capture de changements

Labo : Labo_04.md, étapes 1 et 3.

Relire toute une base chaque nuit ne tient pas longtemps. Capter les changements (*change data capture*) veut dire n'extraire que ce qui a été créé, modifié ou supprimé depuis la dernière fois. Trois méthodes existent, de la plus simple à la plus complète.

3.1 Par requête, sur un filigrane

On retient la plus haute date de modification déjà extraite, le filigrane (*watermark*), et l'on demande la suite. Une commande de l'ERP porte deux dates en texte : ERDAT, sa création, et AEDAT, sa dernière modification, qui vaut 00000000 tant qu'elle n'a jamais été modifiée. Pour décembre 2025, le détail est parlant :

Filtre	Commandes
modifiées en décembre (AEDAT >= '20251201')	37
créées en décembre et jamais modifiées	390
créées avant décembre, modifiées en décembre	9
créées ou modifiées en décembre (ERDAT >= '20251201' OR AEDAT >= '20251201')	427

Un filigrane posé sur AEDAT seul raterait 390 commandes. Posé sur les deux dates, il rend les 427 commandes de l'extrait de décembre, ligne pour ligne : c'est d'ailleurs ainsi que l'ERP fabrique ses extraits mensuels.

Le filigrane a besoin d'un index. La base de l'ERP en porte un sur AEDAT. Le plan de PostgreSQL le montre : un filtre sur AEDAT seul passe par l'index (Index Scan using ix_vbak_aedat), mais le filtre complet, avec son OR sur ERDAT qui n'a pas d'index, lit la table entière (Seq Scan). Sur 25 483 commandes, c'est instantané ; sur des centaines de millions de lignes, c'est l'extraction qui bloque la nuit.

La méthode a des angles morts, qu'il faut connaître avant de s'y fier :

Angle mort	Ce qui se passe	Parade
les suppressions	une ligne supprimée ne revient dans aucune requête	une suppression logique, comme l'indicateur LOEVM des concessionnaires fermés ; sinon, une autre méthode

Angle mort	Ce qui se passe	Parade
une date au jour près	AEDAT n'a pas d'heure : une ligne modifiée plus tard le même jour tombe entre deux extractions	relire le jour du filigrane ($\geq$ plutôt que $>$), et compter sur l'idempotence
les transactions longues et les horloges	une ligne validée après l'extraction, avec une date antérieure au filigrane, passe entre les mailles	une marge de sécurité : recommencer un peu avant le filigrane
un programme qui ne tient pas la date à jour	un traitement en lot qui modifie sans toucher AEDAT	vérifier la règle avec l'équipe de l'ERP, et le prouver par un contrôle

3.2 Par les documents de modification

Certains logiciels tiennent eux-mêmes le registre de ce qui change. Dans SAP, les documents de modification notent, pour les objets configurés, qui a modifié quel champ, quand, par quelle transaction, avec l'ancienne et la nouvelle valeur : CDHDR en porte l'en-tête, CDPOS le détail. Chez Boréal, seules les fiches des concessionnaires sont suivies, et les extraits en livrent 53 : 49 changements de palier (KUKLA) le 2024-07-01, par le programme de révision ZPALIER, et 4 fermetures (LOEVM), saisies à la main par XD02.

Leur force est de garder chaque changement, même quand la fiche a changé encore depuis : le palier d'avant la révision n'existe plus dans KNA1, mais CDPOS le garde. Leur limite est de ne couvrir que ce que l'entreprise a choisi de suivre.

3.3 Par le journal de la base

Toute base transactionnelle écrit chaque changement dans son journal avant de l'appliquer : le *WAL* de PostgreSQL, le *binlog* de MySQL. Un outil de capture, comme Debezium, lit ce journal et en fait un flux d'événements : chaque insertion, modification et suppression, dans l'ordre, sans interroger les tables. C'est la méthode la plus complète et la plus légère pour la base, mais elle demande l'accès au journal, un connecteur à exploiter et un format d'événements à comprendre. La synthèse Comptoir Aurore rejoue un tel journal, de l'instantané initial jusqu'à la RAW.

Méthode	Voit les suppressions	Charge sur la base	Ce qu'elle demande
Filigrane	non	une requête, et un index	une date de modification fiable
Documents de modification	selon la configuration	faible	un logiciel qui les tient
Journal de la base	oui	presque nulle	l'accès au journal et un connecteur

4. Tirer une API REST

Labo : Labo_04.md, étape 4.

Une API REST répond à des requêtes HTTP. Le client demande une adresse, avec ses paramètres ; le serveur répond par un code et un document. L'API d'ECCC, `api.weather.gc.ca`, suit la norme OGC API Features : des collections (`climate-daily`, `climate-hourly`, `climate-stations`), des entités filtrées par propriété (`CLIMATE_IDENTIFIER=7060400`) ou par période (`datetime`), et trois formats de réponse, JSON, CSV et HTML.

Une réponse ne contient jamais tout d'un coup. L'API découpe le résultat en pages :

Élément	Ce qu'il dit	Pour Bagotville, d'octobre à décembre 2025, pages de 50
<code>limit, offset</code>	la taille de la page, et le rang de la première entité	50, puis 0 et 50
<code>numberMatched</code>	le nombre d'entités qui répondent à la requête	92
<code>numberReturned</code>	le nombre d'entités de cette page	50, puis 42
le lien <code>next</code>	l'adresse de la page suivante, absente sur la dernière	présent sur la première page seulement

Le serveur plafonne la taille d'une page. ECCC l'annonce à 10 000 entités : une demande de 15 000 observations horaires de Bagotville rend `numberReturned` à 10 000, sur 37 068 annoncées. Un client qui ne lirait que la première réponse croirait tout avoir, et c'est pourquoi on compare, à la fin, les entités reçues à `numberMatched`.

Les codes HTTP disent quoi faire d'une erreur :

Code	Sens	Réessayer
200	réponse normale	sans objet
400	requête fausse ; ECCC répond <code>InvalidParameterValue</code> pour un <code>limit</code> qui n'est pas un nombre	non : la même requête échouera encore
404	ressource absente ; ECCC répond <code>NotFound</code> pour une collection inconnue	non
429	trop de requêtes : le serveur demande de ralentir, parfois en indiquant le délai par l'en-tête <code>Retry-After</code>	oui, après le délai
5xx	panne du serveur	oui, après un délai
aucun code	coupure du réseau, délai dépassé	oui, après un délai

Le délai double à chaque échec (2, 4, puis 8 secondes dans le labo) : c'est la temporisation exponentielle, qui laisse au serveur le temps de se remettre au lieu de l'assaillir. Le nombre d'essais reste borné, et la dernière erreur remonte. Le Cours 05 fait de ces reprises un outil commun. ECCC n'annonce pas de limite de débit : raison de plus pour rester poli, avec des pages assez grandes et des requêtes l'une après l'autre.

La météo de la RAW vient d'un instantané figé de cette même API, pour que tout le monde obtienne les mêmes chiffres. Le labo tire les 92 jours d'octobre à décembre 2025 et les compare à l'instantané. Comparées en texte, les valeurs semblent différer : 5 températures moyennes et 45 précipitations, parce que l'API rend 6.0 là où l'instantané a écrit 6. Comparées en nombres, elles concordent toutes.

Piège — comparer des nombres en texte. '6.0' et '6' sont deux chaînes différentes et le même nombre. Une comparaison faite dans le mauvais type invente des écarts, ou en cache d'autres. Comparez toujours deux valeurs dans le type de ce qu'elles représentent.

Recoupements — ce que ce cours partage avec ses voisines. - SQL avancé. Les Cours 10 et 11 de SQL possèdent le plan de requête et l'index, la lecture longue et les versions de lignes ; ici, ce qu'ils changent pour qui extrait d'une base.

Points clés à retenir

- Une source qui pousse livre ce qu'elle a ; une source qu'on tire attend la demande. Presque tous les pièges d'une ingestion viennent de ce partage des responsabilités.
 - ODBC fait du pilote un contrat. On lit une base de production par paquets, hors des pointes, avec un compte en lecture.
 - Un filigrane capte les créations et les modifications, pas les suppressions ; il demande une date fiable et un index. Les documents de modification et le journal de la base vont plus loin.
 - Une API se lit jusqu'à la dernière page, avec des reprises bornées sur les erreurs passagères seulement, et l'on compte ce qu'on a reçu.
-

Exercices

Les exercices de ce cours vivent dans Exercices_04.md, et leur corrigé dans Corrige_04.md. Faites-les après le labo : plusieurs énoncés supposent que la base de l'ERP tourne dans son conteneur.